

```

1 //#####
2 // FILE:   LABstarter_main.c
3 //
4 // TITLE:  Lab Starter
5 //#####
6
7 // Included Files
8 #include <stdio.h>
9 #include <stdlib.h>
10 #include <stdarg.h>
11 #include <string.h>
12 #include <math.h>
13 #include <limits.h>
14 #include "F28x_Project.h"
15 #include "driverlib.h"
16 #include "device.h"
17 #include "F28379dSerial.h"
18 #include "song.h"
19 #include "dsp.h"
20 #include "fpu32/fpu_rfft.h"
21
22 #define PI          3.1415926535897932384626433832795
23 #define TWOPI       6.283185307179586476925286766559
24 #define HALFPI      1.5707963267948966192313216916398
25 // The Launchpad's CPU Frequency set to 200 you should not change this value
26 #define LAUNCHPAD_CPU_FREQUENCY 200
27
28
29 // Interrupt Service Routines predefinition
30 __interrupt void cpu_timer0_isr(void);
31 __interrupt void cpu_timer1_isr(void);
32 __interrupt void cpu_timer2_isr(void);
33 __interrupt void SWI_isr(void);
34
35 // sjew: ISR dec
36 __interrupt void ADCC_ISR (void);
37
38 // sjew: fn decs
39 void setEPWM1A(float controleffort);
40 void setEPWM2A(float controleffort);
41 void init_eQEPs(void);
42 float readEncLeft(void);
43 float readEncRight(void);
44 float readEncWheel(void);
45
46 // sjew: adc_isr variables
47 uint32_t ADCC_count = 0;
48 float adcc2result_volts = 0;
49 float adcc3result_volts = 0;
50 float adcc4result_volts = 0;
51 float adcc5result_volts = 0;
52 // sjew: initializations for IMU data
53 float IMUx = 0;
54 float IMU4x = 0;
55 float IMUz = 0;
56 float IMU4z = 0;
57 // sjew: start the offsets at 1.23 because that is what they are assumed to be but then theyre

```

```
    adjusted later
58 float IMU_x_offset = 1.23;
59 float IMU_4x_offset = 1.23;
60 float IMU_z_offset = 1.23;
61 float IMU_4z_offset = 1.23;
62
63 float sum_x = 0;
64 float sum_4x = 0;
65 float sum_z = 0;
66 float sum_4z = 0;
67
68 float IMUz_prev = 0.0;
69 float IMU4z_prev = 0.0;
70
71 float accum_IMUz = 0.0;
72 float accum_IMU4z = 0.0;
73
74 uint8_t adcc_count_flag = 0;
75
76 // sjew: global variables to track velocity, position, effort values
77 int pwm_iterator = 1;
78 int motor_counting_up = 1;
79 float ctrl_effort = 0.0;
80 float curr_left_enc = 0;
81 float curr_right_enc = 0;
82 float curr_p_right = 0;
83 float curr_p_left = 0;
84 float prev_p_right = 0;
85 float prev_p_left = 0;
86 float v_left = 0;
87 float v_right = 0;
88 float curr_knob_enc = 0;
89 float left_rpf = 9.83;
90 float right_rpf = 9.833;
91 float u_left = 0;
92 float u_right = 0;
93
94 // sjew: friction const
95 float visc_pos = 2.3335 * 0.6;
96 float static_pos = 1.8655 * 0.6;
97 float visc_neg = 2.2755 * 0.6;
98 float static_neg = -1.863 * 0.6;
99
100 // sjew: PID Constants
101 float v_ref = 0;
102 float kp = 3.0;
103 float ki = 25.0;
104 float e_left = 0.0;
105 float e_right = 0.0;
106 float accum_e_left = 0.0;
107 float accum_e_right = 0.0;
108 float prev_e_left = 0.0;
109 float prev_e_right = 0.0;
110 float e_steer = 0.0;
111 float kp_turn = 3.0;
112 float turn_command = 0.0;
113
```

```
114 // Count variables
115 uint32_t numTimer0calls = 0;
116 uint32_t numSWIcalls = 0;
117 extern uint32_t numRXA;
118 uint16_t UARTPrint = 0;
119
120
121 void main(void)
122 {
123     // PLL, WatchDog, enable Peripheral Clocks
124     // This example function is found in the F2837xD_SysCtrl.c file.
125     InitSysCtrl();
126
127     InitGpio();
128
129     // Blue LED on LaunchPad
130     GPIO_SetupPinMux(31, GPIO_MUX_CPU1, 0);
131     GPIO_SetupPinOptions(31, GPIO_OUTPUT, GPIO_PUSHPULL);
132     GpioDataRegs.GPASET.bit.GPIO31 = 1;
133
134     // Red LED on LaunchPad
135     GPIO_SetupPinMux(34, GPIO_MUX_CPU1, 0);
136     GPIO_SetupPinOptions(34, GPIO_OUTPUT, GPIO_PUSHPULL);
137     GpioDataRegs.GPBSET.bit.GPIO34 = 1;
138
139     // LED1 and PWM Pin
140     GPIO_SetupPinMux(22, GPIO_MUX_CPU1, 0);
141     GPIO_SetupPinOptions(22, GPIO_OUTPUT, GPIO_PUSHPULL);
142     GpioDataRegs.GPACLEAR.bit.GPIO22 = 1;
143
144     // LED2
145     GPIO_SetupPinMux(94, GPIO_MUX_CPU1, 0);
146     GPIO_SetupPinOptions(94, GPIO_OUTPUT, GPIO_PUSHPULL);
147     GpioDataRegs.GPCCLEAR.bit.GPIO94 = 1;
148
149     // LED3
150     GPIO_SetupPinMux(95, GPIO_MUX_CPU1, 0);
151     GPIO_SetupPinOptions(95, GPIO_OUTPUT, GPIO_PUSHPULL);
152     GpioDataRegs.GPCCLEAR.bit.GPIO95 = 1;
153
154     // LED4
155     GPIO_SetupPinMux(97, GPIO_MUX_CPU1, 0);
156     GPIO_SetupPinOptions(97, GPIO_OUTPUT, GPIO_PUSHPULL);
157     GpioDataRegs.GPDCLEAR.bit.GPIO97 = 1;
158
159     // LED5
160     GPIO_SetupPinMux(111, GPIO_MUX_CPU1, 0);
161     GPIO_SetupPinOptions(111, GPIO_OUTPUT, GPIO_PUSHPULL);
162     GpioDataRegs.GPDCLEAR.bit.GPIO111 = 1;
163
164     // LS7366#1 CS
165     GPIO_SetupPinMux(130, GPIO_MUX_CPU1, 0);
166     GPIO_SetupPinOptions(130, GPIO_OUTPUT, GPIO_PUSHPULL);
167     GpioDataRegs.GPECLEAR.bit.GPIO130 = 1;
168
169     // LS7366#2 CS
170     GPIO_SetupPinMux(131, GPIO_MUX_CPU1, 0);
```

```
171     GPIO_SetupPinOptions(131, GPIO_OUTPUT, GPIO_PUSHPULL);
172     GpioDataRegs.GPECLEAR.bit.GPIO131 = 1;
173
174     // LS7366#3 CS
175     GPIO_SetupPinMux(25, GPIO_MUX_CPU1, 0);
176     GPIO_SetupPinOptions(25, GPIO_OUTPUT, GPIO_PUSHPULL);
177     GpioDataRegs.GPACLEAR.bit.GPIO25 = 1;
178
179     // LS7366#4 CS
180     GPIO_SetupPinMux(26, GPIO_MUX_CPU1, 0);
181     GPIO_SetupPinOptions(26, GPIO_OUTPUT, GPIO_PUSHPULL);
182     GpioDataRegs.GPACLEAR.bit.GPIO26 = 1;
183
184     // WIZNET RST
185     GPIO_SetupPinMux(27, GPIO_MUX_CPU1, 0);
186     GPIO_SetupPinOptions(27, GPIO_OUTPUT, GPIO_PUSHPULL);
187     GpioDataRegs.GPACLEAR.bit.GPIO27 = 1;
188
189     //PushButton 1
190     GPIO_SetupPinMux(157, GPIO_MUX_CPU1, 0);
191     GPIO_SetupPinOptions(157, GPIO_INPUT, GPIO_PULLUP);
192
193     //PushButton 2
194     GPIO_SetupPinMux(158, GPIO_MUX_CPU1, 0);
195     GPIO_SetupPinOptions(158, GPIO_INPUT, GPIO_PULLUP);
196
197     //PushButton 3
198     GPIO_SetupPinMux(159, GPIO_MUX_CPU1, 0);
199     GPIO_SetupPinOptions(159, GPIO_INPUT, GPIO_PULLUP);
200
201     //PushButton 4
202     GPIO_SetupPinMux(160, GPIO_MUX_CPU1, 0);
203     GPIO_SetupPinOptions(160, GPIO_INPUT, GPIO_PULLUP);
204
205     //SPIRAM CS Chip Select
206     GPIO_SetupPinMux(19, GPIO_MUX_CPU1, 0);
207     GPIO_SetupPinOptions(19, GPIO_OUTPUT, GPIO_PUSHPULL);
208     GpioDataRegs.GPASET.bit.GPIO19 = 1;
209
210     //F28027 CS
211     GPIO_SetupPinMux(29, GPIO_MUX_CPU1, 0);
212     GPIO_SetupPinOptions(29, GPIO_OUTPUT, GPIO_PUSHPULL);
213     GpioDataRegs.GPASET.bit.GPIO29 = 1;
214
215     //MPU9250 CS Chip Select
216     GPIO_SetupPinMux(66, GPIO_MUX_CPU1, 0);
217     GPIO_SetupPinOptions(66, GPIO_OUTPUT, GPIO_PUSHPULL);
218     GpioDataRegs.GPCSET.bit.GPIO66 = 1;
219
220     //WIZNET CS Chip Select
221     GPIO_SetupPinMux(125, GPIO_MUX_CPU1, 0);
222     GPIO_SetupPinOptions(125, GPIO_OUTPUT, GPIO_PUSHPULL);
223     GpioDataRegs.GPDSET.bit.GPIO125 = 1;
224
225     // Clear all interrupts and initialize PIE vector table:
226     // Disable CPU interrupts
227     DINT;
```

```
228
229 // Initialize the PIE control registers to their default state.
230 // The default state is all PIE interrupts disabled and flags
231 // are cleared.
232 // This function is found in the F2837xD_PieCtrl.c file.
233 InitPieCtrl();
234
235 // Disable CPU interrupts and clear all CPU interrupt flags:
236 IER = 0x0000;
237 IFR = 0x0000;
238
239 // Initialize the PIE vector table with pointers to the shell Interrupt
240 // Service Routines (ISR).
241 // This will populate the entire table, even if the interrupt
242 // is not used in this example. This is useful for debug purposes.
243 // The shell ISR routines are found in F2837xD_DefaultIsr.c.
244 // This function is found in F2837xD_PieVect.c.
245 InitPieVectTable();
246
247 // Interrupts that are used in this example are re-mapped to
248 // ISR functions found within this project
249 EALLOW; // This is needed to write to EALLOW protected registers
250 PieVectTable.TIMER0_INT = &cpu_timer0_isr;
251 PieVectTable.TIMER1_INT = &cpu_timer1_isr;
252 PieVectTable.TIMER2_INT = &cpu_timer2_isr;
253 PieVectTable.SCIA_RX_INT = &RXAINT_recv_ready;
254 PieVectTable.SCIB_RX_INT = &RXBINT_recv_ready;
255 PieVectTable.SCIC_RX_INT = &RXCINT_recv_ready;
256 PieVectTable.SCID_RX_INT = &RXDINT_recv_ready;
257 PieVectTable.SCIA_TX_INT = &TXAINT_data_sent;
258 PieVectTable.SCIB_TX_INT = &TXBINT_data_sent;
259 PieVectTable.SCIC_TX_INT = &TXCINT_data_sent;
260 PieVectTable.SCID_TX_INT = &TXDINT_data_sent;
261
262 // sjew: adcc_isr_init
263 PieVectTable.ADCC1_INT = &ADCC_ISR;
264
265 PieVectTable.EMIF_ERROR_INT = &SWI_isr;
266 EDIS; // This is needed to disable write to EALLOW protected registers
267
268
269 // Initialize the CpuTimers Device Peripheral. This function can be
270 // found in F2837xD_CpuTimers.c
271 InitCpuTimers();
272
273 // Configure CPU-Timer 0, 1, and 2 to interrupt every given period:
274 // 200MHz CPU Freq, Period (in uSeconds)
275 ConfigCpuTimer(&CpuTimer0, LAUNCHPAD_CPU_FREQUENCY, 10000);
276 ConfigCpuTimer(&CpuTimer1, LAUNCHPAD_CPU_FREQUENCY, 20000);
277 ConfigCpuTimer(&CpuTimer2, LAUNCHPAD_CPU_FREQUENCY, 50000);
278
279 // Enable CpuTimer Interrupt bit TIE
280 CpuTimer0Regs.TCR.all = 0x4000;
281 CpuTimer1Regs.TCR.all = 0x4000;
282 CpuTimer2Regs.TCR.all = 0x4000;
283
284 init_serialSCIA(&SerialA,115200);
```

```

285     init_serialSCIB(&SerialB,19200);
286 //     init_serialSCIC(&SerialC,115200);
287 //     init_serialSCID(&SerialD,115200);
288
289     // sjew: init eQEPs and set up pins to be PWM and not GPIO
290     init_eQEPs();
291     GPIO_SetupPinMux(0, GPIO_MUX_CPU1, 1);
292     GPIO_SetupPinMux(2, GPIO_MUX_CPU1, 1);
293
294     EALLOW;
295     // sjew: motor EPWM Setup
296     // sjew: setup for ePWM1a
297     //     PERIOD: 20KHz, 50% default duty cycle, HIGH on zero and LOW on compare a
298     GpioCtrlRegs.GPAPUD.bit.GPIO0 = 1;
299     EPwm1Regs.TBCTL.bit.FREE_SOFT = 0x2;
300     EPwm1Regs.TBCTL.bit.CLKDIV = 0;
301     EPwm1Regs.TBCTL.bit.CTRMODE = 0;
302     EPwm1Regs.TBCTL.bit.PHSEN = 0;
303     EPwm1Regs.TBCTR = 0;
304     EPwm1Regs.TBPRD = 2500; // 20 KHz (2500/50MHz)
305     EPwm1Regs.CMPA.bit.CMPA = 1250;
306     EPwm1Regs.AQCTLA.bit.CAU = 0x1; // set low
307     EPwm1Regs.AQCTLA.bit.ZRO = 0x2; // set high
308     EPwm1Regs.TBPHS.bit.TBPHS = 0;
309
310     // sjew: setup for ePWM2a
311     //     PERIOD: 20KHz, 50% default duty cycle, HIGH on zero and LOW on compare a
312     GpioCtrlRegs.GPAPUD.bit.GPIO2 = 1;
313     EPwm2Regs.TBCTL.bit.FREE_SOFT = 0x2;
314     EPwm2Regs.TBCTL.bit.CLKDIV = 0;
315     EPwm2Regs.TBCTL.bit.CTRMODE = 0;
316     EPwm2Regs.TBCTL.bit.PHSEN = 0;
317     EPwm2Regs.TBCTR = 0;
318     EPwm2Regs.TBPRD = 2500; // 20 KHz (2500/50MHz)
319     EPwm2Regs.CMPA.bit.CMPA = 1250;
320     EPwm2Regs.AQCTLA.bit.CAU = 0x1; // set low
321     EPwm2Regs.AQCTLA.bit.ZRO = 0x2; // set high
322     EPwm2Regs.TBPHS.bit.TBPHS = 0;
323
324
325     // sjew: epwm trigger for sampling ADC at a 1ms rate
326     EPwm4Regs.ETSEL.bit.SOCAEN = 0; // Disable SOC on A group
327     EPwm4Regs.TBCTL.bit.CTRMODE = 3; // freeze counter
328     EPwm4Regs.ETSEL.bit.SOCASEL = 0x2; // Select Event when counter equal to PRD
329     EPwm4Regs.ETPS.bit.SOCAPRD = 0x1; // Generate pulse on 1st event ("pulse" is the same as
    "trigger")
330     EPwm4Regs.TBCTR = 0x0; // Clear counter
331     EPwm4Regs.TBPHS.bit.TBPHS = 0x0000; // Phase is 0
332     EPwm4Regs.TBCTL.bit.PHSEN = 0; // Disable phase loading
333     EPwm4Regs.TBCTL.bit.CLKDIV = 0; // divide by 1 50Mhz Clock
334     EPwm4Regs.TBPRD = 50000; // Set Period to 1ms sample. Input clock is 50MHz.
335     // Notice here that we are not setting CMPA or CMPB because we are not using the PWM
    signal
336     EPwm4Regs.ETSEL.bit.SOCAEN = 1; //enable SOCA
337     EPwm4Regs.TBCTL.bit.CTRMODE = 0; //unfreeze, and enter up count mode
338     //ADCC
339     AdccRegs.ADCCTL2.bit.PRESCALE = 6; //set ADCCLK divider to /4

```

```

340     AdcSetMode(ADC_ADCC, ADC_RESOLUTION_12BIT, ADC_SIGNALMODE_SINGLE); //read calibration
        settings
341     AdccRegs.ADCCTL1.bit.INTPULSEPOS = 1;
342     AdccRegs.ADCCTL1.bit.ADCPWDNZ = 1;
343     AdccRegs.ADCSOC0CTL.bit.CHSEL = 0x2; //SOC0 will convert Channel you choose Does not have
        to be B0
344     AdccRegs.ADCSOC0CTL.bit.ACQPS = 99; //sample window is acqps + 1 SYSCLK cycles = 500ns
345     AdccRegs.ADCSOC0CTL.bit.TRIGSEL = 0xb; // EPWM4 ADCSOCA or another trigger you choose will
        trigger SOC0
346     AdccRegs.ADCSOC1CTL.bit.CHSEL = 0x3; //SOC1 will convert Channel you choose Does not have
        to be B1
347     AdccRegs.ADCSOC1CTL.bit.ACQPS = 99; //sample window is acqps + 1 SYSCLK cycles = 500ns
348     AdccRegs.ADCSOC1CTL.bit.TRIGSEL = 0xb; // EPWM4 ADCSOCA or another trigger you choose will
        trigger SOC1
349     AdccRegs.ADCSOC2CTL.bit.CHSEL = 0x4; //SOC2 will convert Channel you choose Does not have
        to be B2
350     AdccRegs.ADCSOC2CTL.bit.ACQPS = 99; //sample window is acqps + 1 SYSCLK cycles = 500ns
351     AdccRegs.ADCSOC2CTL.bit.TRIGSEL = 0xb; // EPWM4 ADCSOCA or another trigger you choose will
        trigger SOC2
352     AdccRegs.ADCSOC3CTL.bit.CHSEL = 0x5; //SOC3 will convert Channel you choose Does not have
        to be B3
353     AdccRegs.ADCSOC3CTL.bit.ACQPS = 99; //sample window is acqps + 1 SYSCLK cycles = 500ns
354     AdccRegs.ADCSOC3CTL.bit.TRIGSEL = 0xb; // EPWM4 ADCSOCA or another trigger you choose will
        trigger SOC3
355     AdccRegs.ADCINTSEL1N2.bit.INT1SEL = 0x3; //set to last SOC that is converted and it will
        set INT1 flag ADCB1
356     AdccRegs.ADCINTSEL1N2.bit.INT1E = 1; //enable INT1 flag
357     AdccRegs.ADCINTFLGCLR.bit.ADCINT1 = 1; //make sure INT1 flag is cleared
358     EDIS;
359     // Enable CPU int1 which is connected to CPU-Timer 0, CPU int13
360     // which is connected to CPU-Timer 1, and CPU int 14, which is connected
361     // to CPU-Timer 2: int 12 is for the SWI.
362     IER |= M_INT1;
363     IER |= M_INT8; // SCIC SCID
364     IER |= M_INT9; // SCIA
365     IER |= M_INT12;
366     IER |= M_INT13;
367     IER |= M_INT14;
368
369     // Enable TINT0 in the PIE: Group 1 interrupt 7
370     PieCtrlRegs.PIEIER1.bit.INTx7 = 1;
371     // Enable SWI in the PIE: Group 12 interrupt 9
372     PieCtrlRegs.PIEIER12.bit.INTx9 = 1;
373     PieCtrlRegs.PIEIER1.bit.INTx3 = 1;
374
375     // Enable global Interrupts and higher priority real-time debug events
376     EINT; // Enable Global interrupt INTM
377     ERTM; // Enable Global realtime interrupt DBGEM
378
379
380     // IDLE loop. Just sit and loop forever (optional):
381     while(1)
382     {
383         if (UARTPrint == 1 ) {
384             // Normally on the Robot Car we only use the below UART_printfLine functions to
        write to the
385             // on board LCD screen. This below serial_printf is only used in lab 1 to print

```



```

    to a serial
386         // terminal over a USB cable like you will for your Homeworks.
387         //serial_printf(&SerialA,"Num Timer2:%ld Num SerialRX: %ld\r\n",CpuTimer2.InterruptCount,numRXA);
388
389         //IMPORTANT!! %ld is for an int32_t. To print an int16_t use %d
390         UART_printfLine(1, "z: %.2f, 4z: %.2f", accum_IMUz, accum_IMU4z);
391         UART_printfLine(2, "VR: %.1f, HD: %.1f", v_ref, turn_command);
392 //         UART_printfLine(1,"x: %.2f,4x: %.2f", IMUx, IMU4x);
393 //         UART_printfLine(2,"z: %.2f,4z: %.2f", IMUz, IMU4z);
394 //         UART_printfLine(2, "IMU vals: ")
395         UARTPrint = 0;
396     }
397 }
398 }
399
400
401 // SWI_isr, Using this interrupt as a Software started interrupt
402 __interrupt void SWI_isr(void) {
403
404     // These three lines of code allow SWI_isr, to be interrupted by other interrupt functions
405     // making it lower priority than all other Hardware interrupts.
406     PieCtrlRegs.PIEACK.all = PIEACK_GROUP12;
407     asm("    NOP"); // Wait one cycle
408     EINT;           // Clear INTM to enable interrupts
409
410
411
412     // Insert SWI ISR Code here.....
413
414
415     numSWIcalls++;
416
417     DINT;
418
419 }
420
421 // cpu_timer0_isr - CPU Timer0 ISR
422 __interrupt void cpu_timer0_isr(void)
423 {
424     CpuTimer0.InterruptCount++;
425
426     numTimer0calls++;
427
428 //     if ((numTimer0calls%50) == 0) {
429 //         PieCtrlRegs.PIEIFR12.bit.INTx9 = 1; // Manually cause the interrupt for the SWI
430 //     }
431
432     if ((numTimer0calls%5) == 0) {
433         // Blink LaunchPad Red LED
434         GpioDataRegs.GPBTOGGLE.bit.GPIO34 = 1;
435     }
436
437
438     // Acknowledge this interrupt to receive more interrupts from group 1
439     PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
440 }

```



```

441
442 // cpu_timer1_isr - CPU Timer1 ISR
443 __interrupt void cpu_timer1_isr(void)
444 {
445
446     CpuTimer1.InterruptCount++;
447 }
448
449 // cpu_timer2_isr CPU Timer2 ISR
450 __interrupt void cpu_timer2_isr(void)
451 {
452     // Blink LaunchPad Blue LED
453     GpioDataRegs.GPATOGGLE.bit.GPIO31 = 1;
454
455     CpuTimer2.InterruptCount++;
456
457 // if ((CpuTimer2.InterruptCount % 10) == 0) {
458 //     UARTPrint = 1;
459 // }
460 }
461
462 __interrupt void ADCC_ISR (void){
463     // sjew: get the z, 4z, x, 4x adc values from the gyro and convert to voltages
464     int16_t adcc2result = AdccResultRegs.ADCRESULT0;
465     int16_t adcc3result = AdccResultRegs.ADCRESULT1;
466     int16_t adcc4result = AdccResultRegs.ADCRESULT2;
467     int16_t adcc5result = AdccResultRegs.ADCRESULT3;
468
469     adcc2result_volts = adcc2result * 3.0 / 4095.0;
470     adcc3result_volts = adcc3result * 3.0 / 4095.0;
471     adcc4result_volts = adcc4result * 3.0 / 4095.0;
472     adcc5result_volts = adcc5result * 3.0 / 4095.0;
473     // sjew: convert voltages to deg/sec using the offset
474     IMUx = (adcc2result_volts - IMU_x_offset ) * 400.0;
475     IMU4x = (adcc3result_volts - IMU_4x_offset ) * 100.0;
476
477     IMUz = (adcc5result_volts - IMU_z_offset ) * 400.0;
478     IMU4z = (adcc4result_volts - IMU_4z_offset ) * 100.0;
479
480
481
482     // sjew: print every 500*1ms
483     if ((ADCC_count % 100) == 0) {
484         UARTPrint = 1;
485     }
486     //sjew: if the count is between 1000*1ms and 3000*1ms (1-3s) and the first time its
    reached these values
487     //     then sum the current voltage output from the DAC to take the average
488
489     if(ADCC_count < 1000){
490
491     } else if(ADCC_count < 3000 && adcc_count_flag == 0) {
492         sum_x += adcc2result_volts;
493         sum_4x += adcc3result_volts;
494         sum_z += adcc5result_volts;
495         sum_4z += adcc4result_volts;
496     } else if(ADCC_count == 3000 && adcc_count_flag == 0) {

```

```

497     // sjew: after the first three seconds, take the average of all the offsets and set
the count flag to 1 ( completed )
498     adcc_count_flag = 1;
499     IMU_z_offset = sum_z / 2000.0;
500     IMU_4z_offset = sum_4z / 2000.0;
501     IMU_x_offset = sum_x / 2000.0;
502     IMU_4x_offset = sum_4x / 2000.0;
503 } else {
504     // sjew: integrating for dead reckoning after offsets have been set
505     accum_IMU4z = accum_IMU4z + 0.001 * (IMU4z + IMU4z_prev) / 2.0;
506     accum_IMUz = accum_IMUz + 0.001 * (IMUz + IMUz_prev) / 2.0;
507     IMUz_prev = IMUz;
508     IMU4z_prev = IMU4z;
509 }
510
511 // sjew: control loop
512 curr_left_enc = readEncLeft();
513 curr_right_enc = readEncRight();
514 curr_knob_enc = readEncWheel();
515 // sjew: position calculations
516 curr_p_left = curr_left_enc * 1/ left_rpf;
517 curr_p_right = curr_right_enc * 1/right_rpf;
518 // sjew: vel calculations
519 v_left = (curr_p_left - prev_p_left)/0.001;
520 v_right = (curr_p_right - prev_p_right)/0.001;
521
522
523
524
525 // sjew: control output
526 if(ADCC_count < 4000) {
527     u_left = curr_knob_enc; // sjew: eventually these will be determined by controls
528     u_right = curr_knob_enc;
529 } else {
530     // sjew: error, accumulation, and velocity values
531     v_ref = 1.0;
532     turn_command = -curr_knob_enc / 20.0;
533 //     turn_command = 0.0;
534 //     v_ref = curr_knob_enc / 20.0;
535     e_steer = v_right - v_left + turn_command;
536     e_left = v_ref - v_left + kp_turn * e_steer;
537     e_right = v_ref - v_right - kp_turn * e_steer;
538
539     // sjew: saving temp values for the integral wind up
540     float tmp_accum_left = accum_e_left + ((e_left + prev_e_left) / 2) * 0.001;
541     float tmp_accum_right = accum_e_right + ((e_right + prev_e_right) / 2) * 0.001;
542
543     // sjew: control effort calc
544     u_left = kp * e_left + ki * tmp_accum_left;
545     u_right = kp * e_right + ki * tmp_accum_right;
546
547     // sjew: friction comp
548     if (v_left > 0.0){
549         u_left = u_left + visc_pos* v_left + static_pos;
550     }
551     else{
552         u_left = u_left + visc_neg* v_left + static_neg;

```

```
553     }
554     if (v_right > 0.0){
555         u_right = u_right + visc_pos* v_right + static_pos;
556     }
557     else{
558         u_right = u_right + visc_neg* v_right + static_neg;
559     }
560
561     // sjew: handle integral windup (only accumulate if u_left and u_right are within
    bounds)
562     if(fabs(u_left) < 10.0) {
563         accum_e_left = tmp_accum_left;
564     }
565     if(fabs(u_right) < 10.0) {
566         accum_e_right = tmp_accum_right;
567     }
568     // sjew: always store prev error
569     prev_e_left = e_left;
570     prev_e_right = e_right;
571 }
572
573 // sjew: saturation limits for effort
574 if(u_left < -10.0) {
575     u_left = -10.0;
576 }
577 if(u_left > 10.0) {
578     u_left = 10.0;
579 }
580
581 if(u_right < -10.0) {
582     u_right = -10.0;
583 }
584 if(u_right > 10.0) {
585     u_right = 10.0;
586 }
587
588 // sjew: set pwm outputs
589 setEPWM1A(u_left);
590 setEPWM2A(-u_right);
591 // sjew: store current positions for next interrupt cycle to be used as previous positions
592 prev_p_left = curr_p_left;
593 prev_p_right = curr_p_right;
594
595
596
597 ADCC_count++;
598
599 AdccRegs.ADCINTFLGCLR.bit.ADCINT1 = 1; //clear interrupt flag
600 PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
601 }
602
603 void setEPWM1A(float controleffort) {
604     // sjew: set max and min limits for controleffort
605     if(controleffort < -10.0)
606         controleffort = -10.0;
607     if(controleffort > 10.0)
608         controleffort = 10.0;
```

```

609     int16_t offset = EPwm1Regs.TBPRD/2; // sjew: set an offset that would be a 50% duty cycle
610     float unit_step = EPwm1Regs.TBPRD/20.0; // sjew: unit step of 5% of the total period (each
        1.0 in control effort corresponds to 5% of the total period)
611     EPwm1Regs.CMPA.bit.CMPA = offset+(unit_step*controleffort); // sjew: cmpa to 50% duty
        cycle + (5 * controleffort)% <- controleffort ranges from -10 to 10
612 }
613 void setEPWM2A(float controleffort) {
614     // sjew: set min and max limits for controleffort
615     if(controleffort < -10.0)
616         controleffort = -10.0;
617     if(controleffort > 10.0)
618         controleffort = 10.0;
619     int16_t offset = EPwm2Regs.TBPRD/2; // sjew: set an offset that would be 50% duty cycle
620     float unit_step = EPwm2Regs.TBPRD/20.0; // sjew: unit step of 5% of the total period (each
        1.0 in control effort corresponds to 5% of the total period)
621     EPwm2Regs.CMPA.bit.CMPA = offset+(unit_step*controleffort); // sjew: cmpa to 50% duty
        cycle + (5 * controleffort)% <- controleffort ranges from -10 to 10
622 }
623
624 void init_eQEPs(void) {
625     // setup eQEP1 pins for input
626     EALLOW;
627     //Disable internal pull-up for the selected output pins for reduced power consumption
628     GpioCtrlRegs.GPAPUD.bit.GPIO20 = 1; // Disable pull-up on GPIO20 (EQEP1A)
629     GpioCtrlRegs.GPAPUD.bit.GPIO21 = 1; // Disable pull-up on GPIO21 (EQEP1B)
630     GpioCtrlRegs.GPAQSEL2.bit.GPIO20 = 2; // Qual every 6 samples
631     GpioCtrlRegs.GPAQSEL2.bit.GPIO21 = 2; // Qual every 6 samples
632     EDIS;
633     // This specifies which of the possible GPIO pins will be EQEP1 functional pins.
634     // Comment out other unwanted lines.
635     GPIO_SetupPinMux(20, GPIO_MUX_CPU1, 1);
636     GPIO_SetupPinMux(21, GPIO_MUX_CPU1, 1);
637     EQep1Regs.QEPCTL.bit.QPEN = 0; // make sure eqep in reset
638     EQep1Regs.QDECCTL.bit.QSRC = 0; // Quadrature count mode
639     EQep1Regs.QPOSCTL.all = 0x0; // Disable eQep Position Compare
640     EQep1Regs.QCAPCTL.all = 0x0; // Disable eQep Capture
641     EQep1Regs.QEINT.all = 0x0; // Disable all eQep interrupts
642     EQep1Regs.QPOSMAX = 0xFFFFFFFF; // use full range of the 32 bit count
643     EQep1Regs.QEPCTL.bit.FREE_SOFT = 2; // EQep unaffected by emulation suspend in Code
        Composer
644     EQep1Regs.QPOSCNT = 0;
645     // SE423 8 Lab #3
646     EQep1Regs.QEPCTL.bit.QPEN = 1; // Enable EQep
647     EALLOW;
648     // setup QEP2 pins for input
649     //Disable internal pull-up for the selected output pinsfor reduced power consumption
650     GpioCtrlRegs.GPBPUD.bit.GPIO54 = 1; // Disable pull-up on GPIO54 (EQEP2A)
651     GpioCtrlRegs.GPBPUD.bit.GPIO55 = 1; // Disable pull-up on GPIO55 (EQEP2B)
652     GpioCtrlRegs.GPBQSEL2.bit.GPIO54 = 2; // Qual every 6 samples
653     GpioCtrlRegs.GPBQSEL2.bit.GPIO55 = 2; // Qual every 6 samples
654     EDIS;
655     GPIO_SetupPinMux(54, GPIO_MUX_CPU1, 5); // set GPIO54 and eQep2A
656     GPIO_SetupPinMux(55, GPIO_MUX_CPU1, 5); // set GPIO55 and eQep2B
657     EQep2Regs.QEPCTL.bit.QPEN = 0; // make sure qep reset
658     EQep2Regs.QDECCTL.bit.QSRC = 0; // Quadrature count mode
659     EQep2Regs.QPOSCTL.all = 0x0; // Disable eQep Position Compare
660     EQep2Regs.QCAPCTL.all = 0x0; // Disable eQep Capture

```

```

661     EQep2Regs.QEINT.all = 0x0; // Disable all eQep interrupts
662     EQep2Regs.QPOSMAX = 0xFFFFFFFF; // use full range of the 32 bit count.
663     EQep2Regs.QEPCTL.bit.FREE_SOFT = 2; // EQep unaffected by emulation suspend
664     EQep2Regs.QPOSCNT = 0;
665     EQep2Regs.QEPCTL.bit.QPEN = 1; // Enable EQep
666     EALLOW;
667     // setup QEP3 pins for input
668     //Disable internal pull-up for the selected output pins for reduced power consumption
669     GpioCtrlRegs.GPAPUD.bit.GPIO6 = 1; // Disable pull-up on GPIO54 (EQEP3A)
670     GpioCtrlRegs.GPAPUD.bit.GPIO7 = 1; // Disable pull-up on GPIO55 (EQEP3B)
671     GpioCtrlRegs.GPAQSEL1.bit.GPIO6 = 2; // Qual every 6 samples
672     GpioCtrlRegs.GPAQSEL1.bit.GPIO7 = 2; // Qual every 6 samples
673     EDIS;
674     GPIO_SetupPinMux(6, GPIO_MUX_CPU1, 5); // set GPIO6 and eQep2A
675     GPIO_SetupPinMux(7, GPIO_MUX_CPU1, 5); // set GPIO7 and eQep2B
676     EQep3Regs.QEPCTL.bit.QPEN = 0; // make sure qep reset
677     EQep3Regs.QDECTL.bit.QSRC = 0; // Quadrature count mode
678     EQep3Regs.QPOSCNT.all = 0x0; // Disable eQep Position Compare
679     EQep3Regs.QCAPCTL.all = 0x0; // Disable eQep Capture
680     EQep3Regs.QEINT.all = 0x0; // Disable all eQep interrupts
681     EQep3Regs.QPOSMAX = 0xFFFFFFFF; // use full range of the 32 bit count.
682     EQep3Regs.QEPCTL.bit.FREE_SOFT = 2; // EQep unaffected by emulation suspend
683     EQep3Regs.QPOSCNT = 0;
684     EQep3Regs.QEPCTL.bit.QPEN = 1; // Enable EQep
685 }
686
687 float readEncLeft(void) {
688     int32_t raw = 0;
689     uint32_t QEP_maxvalue = 0xFFFFFFFF; //4294967295U
690     raw = EQep1Regs.QPOSCNT;
691     // SE423 9 Lab #3
692     if (raw >= QEP_maxvalue/2) raw -= QEP_maxvalue; // I don't think this is needed and never
true
693     return -(raw*(1/2000.0*1/20.0*TWOPI));
694 }
695
696 float readEncRight(void) {
697     int32_t raw = 0;
698     uint32_t QEP_maxvalue = 0xFFFFFFFF; //4294967295U -1 32bit signed int
699     raw = EQep2Regs.QPOSCNT;
700     if (raw >= QEP_maxvalue/2) raw -= QEP_maxvalue; // I don't think this is needed and never
true
701     return (raw*(1/2000.0*1/20.0*TWOPI)); // sjew: scaling factor of rotations/counts * wheel
revs/rotation * radians/revolution
702 }
703
704 float readEncWheel(void) {
705     int32_t raw = 0;
706     uint32_t QEP_maxvalue = 0xFFFFFFFF; //4294967295U -1 32bit signed int
707     raw = EQep3Regs.QPOSCNT;
708     if (raw >= QEP_maxvalue/2) raw -= QEP_maxvalue; // I don't think this is needed and never
true
709     return (raw*(2*PI/4000.0)); // sjew: scaling factor of rotations/counts * wheel
revs/rotation * radians/revolution
710 }
711
712

```